

1. BDD (Behavior)

Gherkin (bahasa bisnis)

Bikin ekstensi .feature di simpen di resource

Isinya:

- Fitur: fitur apa yang mau diimplementasi (peminjaman borrower)
- Skenario: kasusnya (borrower bisa meminjam uang ketika terverifikasi)
- Given: inisialisasi/kondisi awal (borrower terverifikasi)
- When: aksi user (borrower meminjam uang)
- Then: hasil (borrower harus approved peminjamannya)

Contoh:

Feature: Pengajuan Pinjaman Borrower

Sebagai Borrower

Saya ingin mengajukan pinjaman

Agar saya bisa mendapatkan dana

Scenario: [TC-01] Pengajuan pinjaman berhasil dengan data valid

Given Borrower dengan ID "BR-001" terdaftar dan tidak di-blacklist

When Borrower mengajukan pinjaman sebesar 5000000

Then Sistem akan membuatkan Loan baru dengan status "PROPOSED"

Scenario: [TC-02] Menolak pinjaman jika nominal nol atau negatif

Given Borrower dengan ID "BR-001" terdaftar dan tidak di-blacklist

When Borrower mengajukan pinjaman sebesar 0

Then Sistem harus menolak pengajuan dengan pesan error

1. Buat gherkin file .feature, contoh (peminjamanborrower.feature)
2. Buat file dengan satu file satu fitur dengan beberapa skenario
3. Jal **mvn test** nanti **Cucumber** akan ngasih template saran step definitionnya di terminal
4. Buat filesteps.java, contoh (peminjamanborrowersteps.java), sesuai dengan file .feature isinya salin hasil dari template no 3
5. Isi template yang ada di filesteps.java dengan logiknya dan harus pake mocking
6. Uji keseluruhan alur di folder application, harus pake mocking objek dan repositorynya (lanjut ke TDD)
7. Uji logic method spesifiknya itu di folder domain (lanjut ke TDD)

2. TDD (Test)

1. Test menggunakan Junit
2. Buat RED (error)
3. Implementasi methodnya di folder yang main supaya GREEN (jalan), asal jalan walau kode masih blm rapih
4. Refactor kodenya

Mockito

Mocking = buat tiruan dependensi class

Gapake mocking
Class Borrower Borrower borrower = new Borrower (...)
Pake mocking pura pura punya objeknya
@mock Borrower borrowermock;

InjectMocks = masukan si objeknya

Misal Ada fitur class loan = butuh objek borrower dan loanservice.
@mock Borrower borrower;
@mock Loanservice loanservice;
//hasil tiruannya masukan ke loan @InjectMocks Loan loan;

Mockito stubbing = buat method dengan output percobaan/set hasilnya, supaya bisa langsung jalan dulu ning di skip

Verifikasiborrower{ Return (true); } Creditscoreborrower{ Return 600; }
Penulisannya: when (borrower.getcreditscore()) .then return 600; when(borrower.verifikasi()) .then return true;

3. DDD (Domain)

Entity/entitas = objek yang punya ID atau bisa beda implementasinya kurg salah

Contoh: Borrower, Lender, Admin, Repayment, Investment

Value object = objek yang gabisa diubah (nilainya tetap/immutable)

Contoh: Money, Tenor, Interestrate (Bunga), Limit

Aggregate = kumpulan entity dan value object yang memiliki hubungan dekat

Contoh: Loan (Borrower, Money, Repayment)

Loan sebagai agregate root jadi yang diluar bakal komunikasi sama
Loan nya gabisa langsung akses borrower atau money atau repayment
langsung

Repository = mini database, satu database untuk satu agregate root (misal :
Loanrepository.java), berupa hashmap, bentuknya interface.

Domain Event = pemberi informasi/notifikasi bahwa ada event yang sudah selesai

DDD Layer:

1. Presentation (tampilan CLI, main.java dll)
2. Application (service, engga ada logic , isinya Cuma sebagai orchestration)
3. Domain (logic)
4. Infrastructure (database in memory)

Design Pattern:

- observer pattern (domain event)
- repository pattern (repository)
- state pattern
- strategy pattern (bunga)

- factory method
- decorator
- chain of responsibility

Bagi Tugas:

1. Peminjamanborrower.feature (faqih)
2. Siklusstatuspeminjaman.feature (arsel)
3. InvestasiLender.feature (darva)
4. PencairanNotifikasi.feature (rajbi)
5. PembayaranCicilan.feature (imam)

Inisialisasi awal:

1. Restruktur folder
2. Isi kerangka awal

Money.java:

```
package com.p2p.domain.valueobject;

import java.math.BigDecimal;

public class Money {
    private final BigDecimal amount;
    private final String currency;

    public Money(BigDecimal amount, String currency) {
        this.amount = amount;
        this.currency = currency; // Default "IDR"
    }

    // TODO (Bersama): Nanti tambahkan method operasi seperti add(),
    subtract(), isGreaterThan() saat TDD

    public BigDecimal getAmount() { return amount; }
    public String getCurrency() { return currency; }
}
```

Borrower.java

```
package com.p2p.domain.borrower;

import com.p2p.domain.shared.Money;

public class Borrower {
    private String id;
    private Money limitPinjaman; // Pakai Money atau class BorrowerLimit

    public Borrower(String id, Money limitAwal) {
        this.id = id;
        this.limitPinjaman = limitAwal;
    }

    // TODO (): Bikin fungsi validasi saat ngurangin limit (TDD Fase
    Pengajuan)
    public void kurangiLimit(Money nominalPinjaman) {
        // Logikanya menyusul
    }

    public String getId() { return id; }
    public Money getLimitPinjaman() { return limitPinjaman; }
}
```

Lender.java

```

package com.p2p.domain.lender;

import com.p2p.domain.shared.Money;

public class Lender {
    private String id;
    private Money saldoBalance;

    public Lender(String id, Money saldoAwal) {
        this.id = id;
        this.saldoBalance = saldoAwal;
    }

    // TODO (): Bikin fungsi buat ngecek saldo cukup atau engga saat
    investasi
    public void kurangiSaldoUntukInvestasi(Money nominalInvestasi) {
        // Logikanya menyusul
    }

    public String getId() { return id; }
}

```

Loan.java (aggregate root)

```

package com.p2p.domain.loan;

import com.p2p.domain.shared.Money;
import java.util.ArrayList;
import java.util.List;

public class Loan {
    // --- ATRIBUT DASAR ---
    private String id;
    private String borrowerId;
    private Money targetNominal;
    private Money totalTerkumpul;

    // TODO (): Nanti refactor String ini jadi State Pattern (misal: LoanState
    status)
    private String status;

    // TODO (): Nanti buat class Repayment dan Strategy Bunga untuk diisi
    ke sini
    // private List<Repayment> daftarCicilan = new ArrayList<>();

    // --- CONSTRUCTOR ---
    // TODO (): Buat Factory Pattern di luar class ini untuk validasi
    pembuatannya
    public Loan(String id, String borrowerId, Money targetNominal) {
        this.id = id;
        this.borrowerId = borrowerId;
    }
}

```

```

        this.targetNominal = targetNominal;
        this.totalTerkumpul = new Money(new java.math.BigDecimal("0"),
"IDR");
        this.status = "PROPOSED";
    }

    // --- METHOD KERANGKA UNTUK TDD MASING-MASING ---

    // TODO (): TDD untuk transisi status (Proposed -> Funding ->
Disbursed)
    public void ubahStatus(String statusBaru) {
        // Jangan diisi dulu, biarkan yang isi saat ngerjain TDD-nya
    }

    // TODO ( & ): TDD untuk investasi masuk
    public void tambahPendanaan(String lenderId, Money
investasiDiberikan) {
        // : Tambahkan nilai ke totalTerkumpul
        // : Bikin if-condition, kalau totalTerkumpul == targetNominal, trigger
Observer (Event)
    }

    // TODO (): TDD untuk hitung cicilan
    public void bayarCicilan(String repaymentId, Money jumlahBayar) {
        // yang akan isi logikanya nanti
    }

    // Getter...
    public String getId() { return id; }
    public String getStatus() { return status; }
}

```

4. Struktur Folder:

